

Flutter Mobile Development

Sesión 01 — Introducción a Flutter y Dart

Luis Dolorier

Tecylab

Agenda

- 1 ¿Qué es Flutter?
- 2 Flutter vs otras tecnologías
- 3 Arquitectura multiplataforma
- 4 Introducción a Dart
- 5 Instalación y primer proyecto

¿Qué es Flutter?

*SDK de desarrollo UI creado por Google para construir aplicaciones **nativas** desde un único código base.*

- Renderizado propio — no depende de componentes del sistema
- Compilación a código nativo (alto rendimiento)
- UI consistente en **todas** las plataformas
- Hot Reload para ciclos de desarrollo rápidos

Ecosistema actual

Plataforma	Soporte
Android	Estable
iOS	Estable
Web	Estable
Windows	Estable
macOS	Estable
Linux	Estable

Flutter vs React Native

Aspecto	Flutter	React Native
Renderizado	Motor propio (Skia)	Componentes nativos
Lenguaje	Dart	JavaScript/TypeScript
Performance	Alto	Medio-alto
Consistencia UI	Alta (siempre igual)	Depende del SO

Flutter ofrece mayor control y consistencia visual.

Flutter vs SwiftUI / Kotlin Multiplatform

Tecnología	Comparte	Para quién
SwiftUI	Solo iOS/macOS	Ecosistema Apple
Kotlin Multiplatform	Lógica	Equipos Android/JVM
Flutter	UI + lógica	Multiplataforma real

*Flutter es la única opción que unifica **UI y lógica** en todas las plataformas.*

Arquitectura de Flutter

Tres capas:

- **Framework** — Widgets, animaciones, gestos
- **Engine** — Renderizado con Skia/Impeller, acceso a plataforma
- **Embedder** — Android, iOS, Web, Desktop

¿Por qué importa?

Flutter controla el píxel — la UI es idéntica en todos lados.

Lenguaje de programación creado por Google, optimizado para UI.

Características clave:

- Tipado estático con inferencia
- Orientado a objetos
- Compilación JIT (desarrollo) y AOT (producción)
- Null Safety nativo
- Asincronía de primera clase

Variables y tipos

// Inferencia de tipos

```
var name = 'Luis';
```

```
var year = 2026;
```

// Tipos explícitos

```
String language = 'Dart';
```

```
int age = 30;
```

```
double rating = 4.5;
```

```
bool active = true;
```

// Inmutabilidad

```
final country = 'Peru'; // runtime constant
```

```
const pi = 3.14159; // compile-time constant
```

Funciones

// Forma completa

```
String greet(String name) {  
  return 'Hola, $name!';  
}
```

// Forma corta (arrow)

```
String greet(String name) => 'Hola, $name!';
```

// Parámetros nombrados y opcionales

```
void register({required String email, String? role}) {  
  // ...  
}
```

Classes

```
class User {  
  final String name;  
  final int    age;  
  
  User(this.name, this.age);  
  
  // Constructor nombrado  
  User.guest() : name = 'Guest', age = 0;  
  
  // Getter  
  String get info => '$name ($age años)';  
}
```

Null Safety

El problema: errores por valores null en tiempo de ejecución.

La solución en Dart:

```
String name    = 'Luis'; // nunca null
String? company = null;   // puede ser null
```

// Operadores de null safety

```
print(company?.toUpperCase()); // safe call
```

```
print(company ?? 'Sin empresa'); // fallback
```

El compilador garantiza que name nunca sea null.

Programación asíncrona — Future

```
Future<String> fetchUser() async {  
  // simula llamada a API  
  await Future.delayed(Duration(seconds: 1));  
  return 'Luis Dolorier';  
}  
  
void main() async {  
  final user = await fetchUser();  
  print(user); // Luis Dolorier  
}
```

Programación asíncrona — Stream

```
Stream<int> counter() async* {  
  for (int i = 1; i <= 5; i++) {  
    await Future.delayed(Duration(seconds: 1));  
    yield i;  
  }  
}
```

// Escuchar el stream

```
counter().listen((value) => print(value));
```

*Un Stream es un Future que emite **múltiples valores** en el tiempo.*

Instalación del entorno

Requisitos:

- Flutter SDK
- Visual Studio Code + extensión Flutter
- Android Studio (emuladores Android)
- Git

Verificar instalación:

```
flutter doctor -v
```

```
[x] Flutter (Channel stable)
[x] Android toolchain
[x] Xcode (macOS only)
[x] VS Code (extension installed)
[x] Connected device
    Resolver todos los [!] antes de continuar.
```


Primer proyecto

```
flutter create my_app  
cd my_app  
flutter run
```

Estructura esencial:

```
lib/  
  main.dart    ← punto de entrada  
android/  
ios/  
web/
```

Primera aplicación

```
import 'package:flutter/material.dart';

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: Text('Mi primera app')),
        body: Center(child: Text('¡Hola Flutter!')),
      ),
    );
  }
}
```

Concepto clave: Todo es un Widget

*En Flutter, **cada elemento de la UI es un widget** — desde un texto hasta una pantalla completa.*

- Text, Icon, Image
- Row, Column, Stack
- Scaffold, AppBar, Navigator

La UI se construye **componiendo** widgets, no heredando de vistas.

Ejercicio práctico

Crea una app que muestre:

- Tu nombre
- Tu profesión o rol
- Un botón que imprima un mensaje en consola

Tip: usa Scaffold → Column → Text + ElevatedButton

Lo que aprendimos hoy:

- Qué es Flutter y por qué usarlo
- Comparativa con otras tecnologías
- Fundamentos de Dart
- Null safety y asincronía
- Setup completo + primera app

Próxima sesión: Widgets, layouts y manejo de estado.